

1 Understanding Database Structure

22 May 2026 17:11

111

Understanding Database Structure

Chapter Overview

- » Understanding the language databases use
- » How a relational database functions
- » Data types
- » Relational database management systems (RDBMSs)
- » SQLite

When learning any new technical skill, you need to know the basic vocabulary to start your journey. We aim to find the right balance: arming you with the fundamental terms and concepts you will need for the remainder of this book, while avoiding unnecessary jargon or advanced concepts. In this chapter, we will introduce the concept of a relational database and showcase the types of data you will encounter in a typical database. We will also introduce the fundamental SQL query: the SELECT statement.

Fundamental Terminology

A "datum" is defined as "a piece of information."⁷ **Data** is simply the plural form of datum. Data appears everywhere and is contained in everything, but for practical purposes the term "data" generally refers to

recorded or recordable information. One of the simplest tools used to record and visualize data is the **table**. A table is merely a two-dimensional grid consisting of rows and columns.

When used in a database, a table may also be referred to as a "base relvar," though in this book we will adhere to the term "table." Please see the Terminology Summary graphic (Figure 5).

↓ GRAPHIC fig. 2

This is a table

UserID	Name	DateOfBirth	Height	Weight	BloodType	PrimaryCareDoctor
92463	Archibald Kennedy	08/24/1976	75	310	B-	Dr. Waynewright
92423	Dennis McGhee	03/12/1982	68	190	B+	Dr. Murphy
92436	Cynthia Owens	09/30/1955	60	104	O+	Dr. Waynewright

These are data

As you can see in Figure 2, the table contains different types of data. Data can be names, numbers, dates, characters (like "+" or "-"), or it may be presented in a multitude of other formats. Data, in its purest sense, is just information. Therefore, when handling data it behooves us to constrain it

appropriately. Look at the table in Figure 2; it appears to store basic information about a group of medical patients. Data about the patients is defined using various formats. There are numbers, names, and dates, and in the BloodType field, a string of two characters is used (a letter and a "+" or "-" character). The formats used to render data are not random. All databases contain something known as *metadata*, which is data that describes the structure and formatting of the data itself, commonly referred to as "data about the data." For example, the DateOfBirth field may contain metadata that constricts information in the field into mm/dd/yyyy format. The metadata in the Height field might limit data to two digits in length and require that it be expressed in terms of inches.

The term *database* can be simply defined as a collection of data arranged for ease and speed of search and retrieval by a computer. The database is often symbolized graphically as a multitiered cylindrical icon (Figure 3) meant to symbolize the stacking of hard disks one on top of another to create a high-capacity data storage center.

Data inside the database is typically stored in a collection of tables. Each table contains a specific set of data, which may relate to and reference other data from other tables within the database.

✓ EXAMPLE

The patient data table in Figure 2 is just a table, not a database. It could, however, be incorporated into a database alongside other tables, such as those storing information about lab tests, prescription drugs, appointment histories,

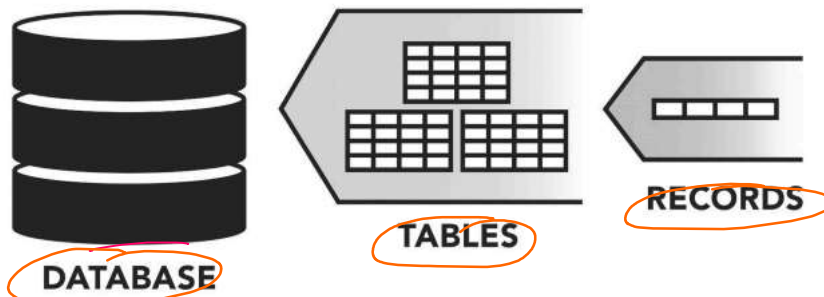
hospital personnel, doctor information, specialties, and appointment availability.

↓ GRAPHIC fig. 3



The purpose of the database is to facilitate the interaction, organization, and analysis of related data across a multitude of sources. When data is placed in tables that have the capacity to relate to one another within a database, a new level of versatility becomes possible.

↓ **GRAPHIC** fig. 4



A database is composed of tables, and tables are composed of records.

The rows in a given table are considered records. They may also be referred to as tuples. The columns in a table may generally be referred to as fields. They may also be called attributes. Fields/attributes are the categories used to define the data within the record (row).

NOTE

Throughout this book we will be using the term "records" to describe the rows in a table and "fields" to describe the columns. Please refer to the Terminology Summary in Figure 5.

↓ **GRAPHIC** fig. 5

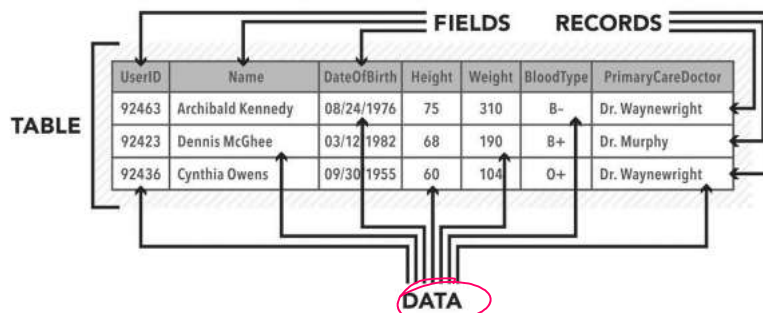
TERMINOLOGY SUMMARY

<u>Terms we will use throughout this book:</u>	<u>May be referred to elsewhere as:</u>
<u>Record</u> Row	<u>Tuple</u>

Field, Column	Attribute
Table	Relation, Base Relvar

Every record is broken down into several fields that represent single elements of data describing a specific thing. For example, our table featured in Figure 6 stores information about patients, presumably at a particular hospital or medical practice or in a particular insurance pool. Whatever the nature of the organization, if they keep a database, then that database will likely be composed of several tables. Understanding how tables refer to and relate to one another is key to understanding essential database architecture.

↓ **GRAPHIC** fig. 6



Fundamental Elements of Relational Databases

The relational database is a database design that was formally conceived of in 1969 by a computer scientist at IBM named Edgar F. Codd. Codd published an article the following year called "A Relational Model of Data for Large Shared Data Banks."⁸ Nine years later, several big players in tech, including IBM and Relational Software Inc. (later to become Oracle), began using relational databases for commercial purposes. Four decades later, the relational model continues to prevail as the most common form of database design.

To gain a rudimentary understanding of how relational databases function, it is important to understand the role of key fields.

↓ GRAPHIC fig. 7

PRIMARY KEY FIELD **FOREIGN KEY FIELD**

PatientID	PatientName	PrimaryCareDoctorID	PrimaryCareDoctorName	DateOfBirth	Height	Weight	BloodType
92463	Archibald Kennedy	106547	Dr. Waynewright	8/24/1976	75	310	B-
92425	Dennis McGhee	106474	Dr. Murphy	3/12/1982	68	190	B+
92443	Cynthia Owens	106547	Dr. Waynewright	9/30/1955	60	104	O+
92478	William Hampton	106437	Dr. Salazar	6/5/1973	73	175	AB-
92392	Hilda Bass	106783	Dr. Dean	6/10/1997	68	152	B+
92436	Frankie Stone	106437	Dr. Salazar	5/28/1979	68	106	O+
92403	Verna Sullivan	106984	Dr. Conner	7/17/2010	66	125	O+
92398	Merle Doyle	106439	Dr. Frank	1/8/1962	65	143	B-
92442	Ruth Swanson	106954	Dr. Hines	2/15/1970	61	160	O-
92384	Johnathan Singleton	106474	Dr. Murphy	6/2/1970	61	232	AB+
92405	WM Patrick	106439	Dr. Frank	6/11/1955	69	196	O+
92376	Mona Norris	106984	Dr. Conner	10/15/1932	60	98	B+
92399	Rick Gordon	106366	Dr. Hart	1/25/2002	68	149	B+
92408	Don Rivera	106437	Dr. Salazar	7/26/1954	72	185	A-
92389	Sheri Griffin	106211	Dr. Harvey	12/16/1987	78	132	AB-
92466	Guillermo Lawrence	106954	Dr. Hines	2/8/1978	60	219	O+
92310	Felipe Parker	106474	Dr. Murphy	12/10/1998	61	165	O-
92413	Brandi Carlson	106399	Dr. Flowers	11/20/2000	66	112	B+
92398	Floyd Casey	106783	Dr. Dean	12/14/1986	61	203	A-
92439	Patrick Walton	106366	Dr. Hart	8/11/1973	76	189	O+
92421	Vicki Klein	106954	Dr. Hines	11/28/1980	65	98	O+
92381	Cathy Harrison	106474	Dr. Murphy	11/16/1946	78	203	AB-
92393	Ann Guerrero	106783	Dr. Dean	6/25/1974	61	142	B-
92437	Gustavo Bates	106399	Dr. Flowers	2/25/2001	78	165	A-

A relational database will contain a multitude of tables similar to the *patient_info* table shown in Figure 7. These tables relate to one another on the basis of key fields. In the *patient_info* table you will notice the primary key field and the foreign key field. As a matter of best practice, every table

in a relational database should have a primary key. The primary key acts as the unique identifier for a record in the table. Each record's primary key must be unique, record by record, and must not be null (empty). Notice the PatientId field in the patient_info table. Since this field is used as the table's primary key, each and every record in the table must have unique data stored in this field. In other words, no two records may contain the same PatientId data.

Although there must be unique data in the primary key field (PatientId in this case), the other fields contain data that may be replicated in more than one record. For example, consider the PrimaryCareDoctorId field; if Dr. Waynewright, ID# 106547 (see the first row in Figure 7), treats multiple patients in the database, then his name and ID may show up in multiple records in the table.

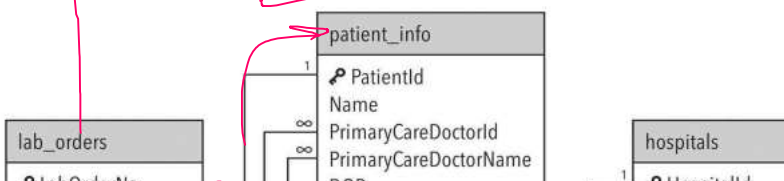
NOTE

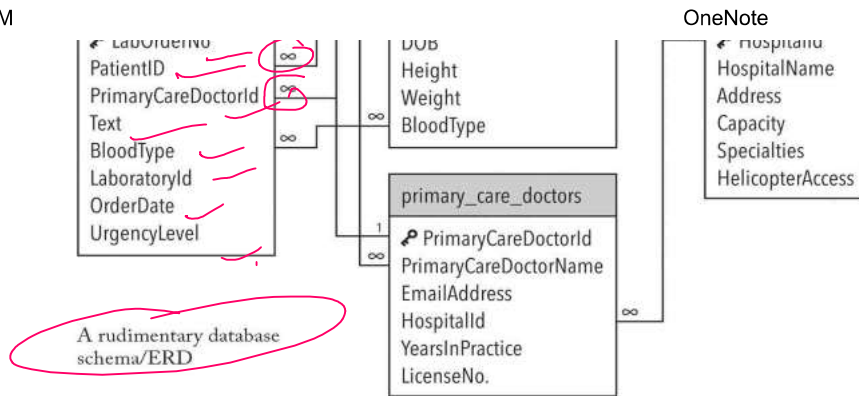
In a relational database, tables are often referred to as "relations," because they contain a set of records (rows) related to various fields (columns). Throughout this book, however, we will be using the term "table." Please refer to the Terminology Summary in Figure 5.

A foreign key field is a field within a table that is acting as a primary key in another table in the database. Let's suppose that in addition to our patient_info table there is another table in the database called primary_care_doctors, which uses the PrimaryCareDoctorId field as its primary key. In the primary_care_doctors table, Dr. Waynewright, with the ID# 106547, will appear in only one record. It is the overlap of various key

fields among tables that facilitates the all-important relatability within the aptly named relational database. These relationships are commonly visualized using a database schema, also known as an entity relationship diagram (ERD), which serves as a kind of blueprint for the database.

GRAPHIC fig. 8

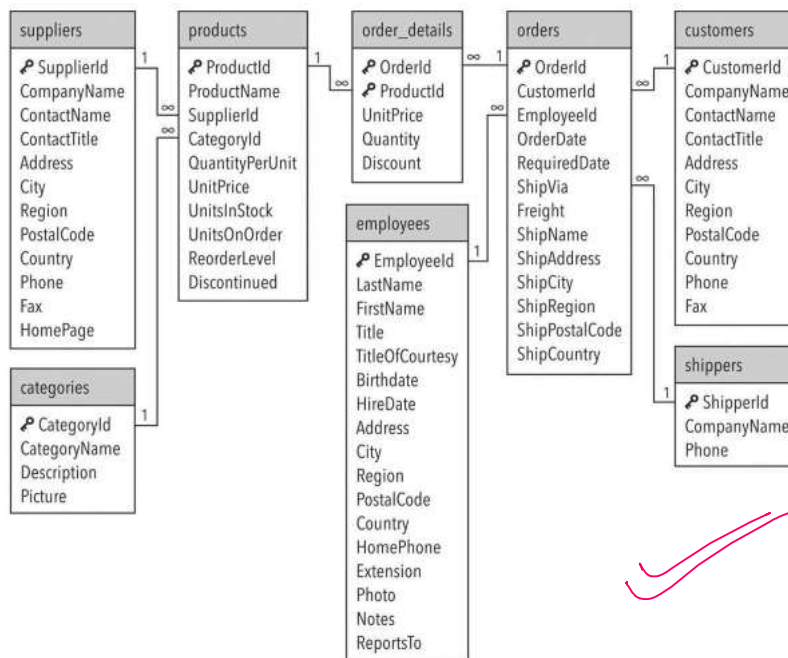




For the moment, don't worry about the 1's and the ∞ symbols. We will explain them in a moment. For now, take a few minutes to study the schema and consider its relationships. There are only four tables in this schema, and the tables are connected to one another by way of one or more common fields. The PatientId field is the primary key for the patient_info table, but it is a foreign key field for the lab_orders table.

Similarly, the HospitalId field is the primary key for the hospitals table, but it is a foreign key field for the primary_care_doctors table. Pretty simple, right? Let's take a look at another schema for a different kind of operation.

GRAPHIC fig. 9



The schema in Figure 9 describes a database that handles the ordering and shipping of products to customers. Now, about those 1 and ∞ symbols found at the ends of the connecting lines in our sample schemas: these

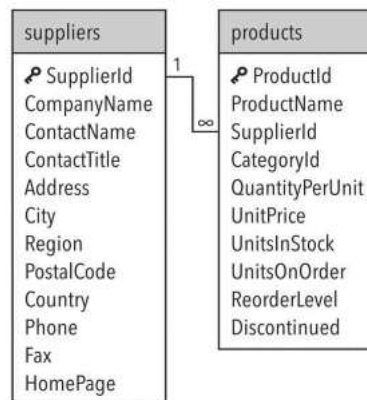
notations describe the nature of the interactivity between tables. When there is a 1 on one end of the connecting line and a ∞ symbol on the other end, it represents a "one-to-many" relationship between the tables' shared fields.

Let's take a closer look at the *products* table (Figure 10). Clearly, the data here is about various products and their attributes.

The *ProductId* field is the primary key for this table, as symbolized by the key icon. Each record in the table will contain a unique product identification number. In fact, that is the whole purpose of this table—to catalog the attributes of the database's various products.

Now, let's look at the relationship between *products* and *suppliers*.

↓ GRAPHIC fig. 10



↓ GRAPHIC fig. 11

SUPPLIERS



SupplierId	CompanyName	ContactName	ContactTitle	Address	City	etc.
S-101	Van Eck Industries	Bruce Davidson	Vp Operations	2158 Del Dew Drive	Temple Hills	...
S-102	Wright & Gate Co	Wilma Joy	Supply Chain Supervisor	291 Creekside Lane	Ventura	...
S-103	Olivias Supply	Brad Pence	Site Manager, Baton Rouge	4353 Locust View Drive	Baton Rouge	...
S-104	Cantor Corporation	Orville Bedford	President	2811 West Drive	Chicago	...
S-105	Bellagio Finland	Wallace Grim	Distributions Supervisor	4939 Breezewood Court	Chanute	...
S-106	Decks Materials	John Tuck	VP Operations	4529 Counts Lane	Lexington	...
S-107	Lennor Co	Rachel Durst	Site Manager, Jackson	2216 Rhapsody Street	Gainesville	...



PRODUCTS



ProductId	ProductName	SupplierId	CategoryId	QuantityPerUnit	UnitPrice	etc.
P001	Welding goggles	S102	SA-432	1	\$12.99	...
P002	Welding helmet	S102	SA-432	1	\$41.49	...
P003	Stick electrodes	S104	WE-214	40	\$7.00	...
P004	Magnetic clamp	S101	WE-220	1	\$11.86	...
P005	Heat resistant blanket	S104	WE-212	1	\$3.73	...
P006	Work table	S105	GE-100	1	\$1,386.67	...
P007	Replacement plates	S105	GE-100	1	\$396.00	...
P008	Welding wire	S104	WE-214	1	\$112.86	...
P009	Welding coveralls	S102	SA-435	1	\$60.27	...
P010	Welding nozzle	S103	WE-214	1	\$141.65	...
P011	Gas regulator	S106	AU-100	1	\$166.25	...
P012	Welding hoods	S102	SA-432	1	\$42.37	...
P013	Spot welding electrode	S104	WE-212	1	\$2.35	...
P014	Plasma cutter	S107	PL-100	1	\$1,645.91	...
P015	Plasma cutter cutting tip	S107	PL-100	1	\$9.27	...

There is a one-to-many relationship between *suppliers* and *products* based on the data in the *SupplierId* field. In *suppliers* each record will possess a unique identification number for each supplier, but in *products* there may be several records with the same supplier identification number.

The key icon next to the *SupplierId* field in *suppliers* alerts us to the fact that *SupplierId* is the primary key for that table. We can certainly have many different products (each with its own unique product ID number) coming from the same supplier and cataloged in the *products* table. Contrast this with *suppliers*, where we must have one and only one unique supplier ID number for each record.

↓ GRAPHIC fig. 13

PRODUCTS

ProductId	ProductName	SupplierId	CategoryId	QuantityPerUnit	UnitPrice	etc.
P001	Welding goggles	S102	SA-432	1	\$12.99	...
P002	Welding helmet	S102	SA-432	1	\$41.49	...
P003	Stick electrodes	S104	WE-214	40	\$7.00	...
P004	Magnetic clamp	S101	WE-220	1	\$11.06	...
P005	Heat resistant blanket	S104	WE-212	1	\$3.73	...
P006	Work table	S105	GE-100	1	\$1,386.67	...
P007	Replacement plates	S105	GE-100	1	\$396.00	...
P008	Welding wire	S104	WE-214	1	\$132.80	...
P009	Welding coveralls	S102	SA-435	1	\$60.27	...
P010	Welding nozzle	S103	WE-214	1	\$161.65	...
P011	Gas regulator	S106	AU-100	1	\$166.25	...
P012	Welding hoods	S102	SA-432	1	\$42.37	...
P013	Spot welding electrode	S104	WE-212	1	\$2.35	...
P014	Plasma cutter	S107	PL-100	1	\$1,645.91	...
P015	Plasma cutter cutting tip	S107	PL-100	1	\$9.27	...

SUPPLIERS

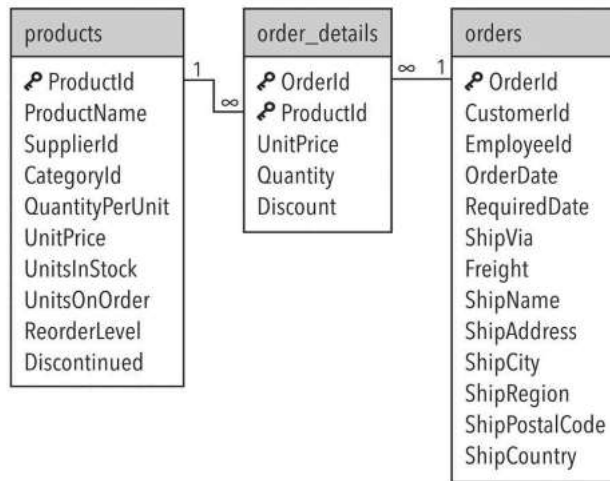
SupplierId	CompanyName	ContactName	ContactTitle	Address	City	etc.
S-101	Van Eck Industries	Bruce Davidson	Vp Operations	2158 Del Dene Drive	Temple Hills	...
S-102	Wright & Gate Co	Wilma Jay	Supply Chain Supervisor	291 Creekside Lane	Venura	...
S-103	Olivias Supply	Brad Pence	Site Manager, Baton Rouge	4353 Lacost View Drive	Baton Rouge	...
S-104	Carroll Corporation	Drville Bedford	President	2811 West Drive	Chicago	...
S-105	Bellagio Finland	Wallace Grim	Distributions Supervisor	4929 Breenwood Court	Chenute	...
S-106	Decks Materials	John Luck	VP Operations	4529 Courts Lane	Lexington	...
S-107	Lemmer Co	Rachel Durg	Site Manager, Jackson	2216 Rhoadsly Street	Guinessville	...

NOTE

Identical supplierId data may appear in multiple records in the products table but not in the suppliers table.

Next, let's look at the relationship between the products, order_details, and orders tables (Figure 14).

↓ **GRAPHIC** fig. 14



The order_details table appears to have two primary keys, as signified by the key icons. It is helpful to think of this scenario as a composite primary key, or composite key, whereby two or more fields are used to define the primary key. Though there are technically two keys in play, it is still best thought of as a single item — the primary key.

↓ **GRAPHIC** fig. 15

COMPOSITE KEY



OrderId	ProductId	UnitPrice	Quantity	Discount
101	P006	\$1,386.67	1	NULL
101	P003	\$7.00	3	NULL
101	P005	\$3.73	1	10%
102	P011	\$166.23	1	NULL
102	P013	\$2.35	1	NULL
103	P014	\$1,645.91	1	NULL
104	P001	\$12.99	3	NULL
104	P012	\$42.37	3	NULL
104	P011	\$166.23	2	10%
104	P003	\$7.00	5	NULL
105	P010	\$141.65	1	NULL
105	P004	\$11.86	3	NULL
105	P003	\$7.00	2	NULL
106	P014	\$1,645.91	1	NULL

The combination of data that populates the fields used to form a composite key acts as the unique identifier for any given record within the table. In other words, if the OrderId of a record in the *order_details* table is "101" and the ProductId for the same record is "P006," then we can assume that no other record in the table will have the same combination of data in those two fields. There may be one or several other records with an OrderId of "101," and there may one or several other records with a ProductId of "P006," but only one record may have both "101" as its OrderId and "P006" as its ProductId. This combination of data across fields acts as the composite key, which, like any primary key, provides a

unique identifier for each and every record in the table.

You may have noticed that the standard primary key in any given table usually represents the "one" in a one-to-many relationship. For example, in *orders*, we can see that the primary key, the OrderId field, will provide a unique identifier for each record in the table. The "many" side of the relationship is found in *order_details*. Why do you think that is?

Let's think this through logically. We can infer that the purpose of *order_details* is to provide information about various products that are ordered. We can infer that any given product may be ordered multiple times by multiple customers under many different circumstances, with different prices, etc. Hence, ProductId cannot be used on its own as a primary key for *order_details*. We may also infer that any given order may include multiple products, and if we look at the other fields used in *order_details*—UnitPrice, Quantity, and Discount—then we can see clearly that these fields speak to the properties of an individual product, not an order at large. Therefore, OrderId cannot be used on its own as a primary key for *order_details*. The solution is to combine ProductId and OrderId into a composite key, thereby ensuring that the data contained in the

UnitPrice, Quantity, and Discount columns corresponds to a unique and specific order and a unique and specific product within that order.

Types of Data

Earlier in this chapter we introduced the concept of metadata, which is data that describes limitations or formatting specifications for other data within the database. When developing a database using SQL, a specific data type must be designated for each and every column used. Data types will vary slightly depending on the version of SQL you are using. In

general, though, you will have numeric data types, character or text-based data types, dates and times, and Boolean values. Let's talk a little bit about each of these.

Numeric Data Types

Numeric data includes integers, which are whole numbers that do not use decimals. Usually, when an integer data type is used, it comes with some form of limitation on its length. Recall that in Figure 7, the table contained data about various patients. For the Weight column we might consider assigning an integer data type with a limit of three digits. Why? Because a) we are okay rounding up or down to the nearest pound or kilogram and do not need to use decimal places, so integers will do. And b) it is inconceivable that we would need more than three digits to describe someone's weight in pounds. When integer data will not do, and we need a more precise numeric format, we can use decimal data, which will allow us to use decimals as needed to fine-tune our numeric values. Like integer data, decimal data can also be length-restricted.

↓ **GRAPHIC** fig. 16

NUMERIC DATA

INTEGER	DECIMAL
5	30.5
6176	14.65
47261	5.634
531	365.1
90	0.437
1	15347.45

 NOTE

Data types that permit longer spans of digits, characters, etc., require more bytes of storage space. SQL also allows monetary numeric data types.

Character or Text-Based Data Types

Character or text-based data types may be configured to hold both a fixed-length string of characters and a variable-length string. For example, if one of your database columns included standard six-character Canadian postal codes (which include both numbers and letters), then you would use a character or text-based data type configured for a fixed-

length string of six characters. If you were creating a column to hold a customer's first or last name, then you would want to configure a variable-length string using reasonable maximum and minimum length limits.

↓ **GRAPHIC** fig. 17

CHARACTER and TEXT-BASED DATA

--	--	--

CanadianZipCode	FirstName	LastName
L4K8R3	Ronald	Dalton
V0S0N2	Clara	Abramson
H7L9N0	Joseph	Scalia
L3M0L7	Benjamin	Dreadnaught
E6K5T8	Harold	Mercedes
E7K3C5	James	Rockefeller

NOTE

The examples we have used thus far have featured relatively short text-based data, such as names and address information. Many databases contain text-based fields that permit much lengthier strings of text and characters. Some database structures can allow for the text of multipage memorandums or even books to be cataloged.

Dates and Times

Dates and times are obviously important data in many circumstances. SQL allows users to choose from a variety of different date-and-time layouts: YYYY-MM-DD, YYYY-MM-DD HH:MI:SS, YY-MM-DD. You may also format a column to hold just the year, either in a four- or two-digit format; that is, "2019" or just "19." Figure 18 exemplifies date and time data in use.



GRAPHIC fig. 18

DATE and TIME DATA

DateOfBirth	CreditCardExpiration	TimeOfDelivery
01/25/1977	08/2023	2019-04-21 08:25:55
09/30/2003	05/2025	2020-12-05 13:30:15
08/15/1999	01/2023	2020-05-10 22:20:36
02/25/1962	11/2022	2019-01-17 10:20:01
09/12/1998	05/2026	2021-06-29 15:21:59
11/02/1950	03/2023	2022-09-03 14:12:24

 NOTE

Date/time formats in SQL have built-in numeric values that allow the

database to interpret requests for chronologically specific outputs. For example, if you want to know how many customers bought a certain product between the dates of October 1, 2020, and December 31, 2020, then SQL can help you generate and sort this output.

Boolean Values

A **Boolean** value is data expressed as either True or False. If you are in charge of a sensitive operation for a government or private entity, then you may use a database to help you keep track of your staff members' security clearance levels. If you need to locate a list of staff members who have security clearances A, B, and D, but not necessarily C, then using Boolean data analysis can make life easier. Figure 19 shows Boolean data in use.

 NOTE

Different versions of SQL will have different lists of recognizable data types. Some versions of SQL, such as SQL Server and MySQL (discussed later in this chapter), do not provide the user with the option to label a data type as "Boolean." Instead, they provide a "Bit" data type, which can easily be appropriated into a quintessential Boolean format.

↓ **GRAPHIC** fig. 19

BOOLEAN DATA

ClearedForTakeOff	InDefault	ConvictedFelon
True	False	False
False	True	False
False	False	False
True	True	True
False	True	False

Relational Database Management Systems

SQL operates in a wide variety of software packages known as **relational database management systems (RDBMSs)**. These systems facilitate the application of SQL when one is issuing commands and posing questions to a database. Popular RDBMS software includes Oracle Database, Microsoft SQL Server, MySQL, IBM Db2, and SQLite.

↓ **GRAPHIC** fig. 20



📄 **NOTE**

It is not uncommon for the RDBMS software itself to be referred to as a database. This is a slight misnomer. More precisely stated, the RDBMS provides an interface (usually known as an SQL browser) for the user to interact with the data stored on the database.

Some RDBMSs are primarily graphic by design. Others are more text-based. RDBMSs also vary in their approach to SQL. We referenced one such anomaly previously in this chapter with regard to the handling of Boolean data. RDBMSs do vary in the way they present database information.

The fact that we are telling the RDBMS what information to present to us defines SQL as a "declarative" programming language. This stands in contrast to other programming languages you may be familiar with, such as C++, Java, etc. Those languages are more procedural, in that they handle creating and running a program from start to finish (allocating memory, including existing reference files, etc.). With SQL, all of the memory

allocation and other procedural duties are handled by the RDBMS.

→ The SELECT Statement

As you know, SQL stands for structured query language, and for several decades it has set the standard for how we communicate with relational databases. The most common SQL command is **SELECT**, a command we will be working with quite closely in chapter 4 and throughout the rest of this book. An SQL query is usually comprised of the **SELECT** keyword in combination with other SQL keywords and references to the data involved in the query. As is the case in other programming languages, the correct sequence and choice of SQL keywords is vital to creating a query that can be correctly interpreted by the SQL browser. This mandated structure is also known as the **syntax** of a query.

In the following example, we can see how syntax of a query varies slightly from one RDBMS implementation to another. These are two very basic queries that essentially do the same thing (they return the first ten records from the *Products* table) but as you can see, they are phrased slightly differently.

In SQL Server we would type the following:

```
SELECT TOP 10 *
FROM
    products;
```

But in MySQL, we would type

```
SELECT *
FROM
    products
LIMIT 10;
```

If we were to structure a query in MySQL similar to the SQL Server example, the SQL browser would generate a syntax error which would prevent the query from running. In this case, the only difference between these two SQL implementations is the way we tell the SQL browser to limit our results to the top ten. The rest of the query is the same. The variations between RDBMSs are generally very minor, usually less than a 10 percent change from one to another. The simple, declarative nature of SQL is fairly consistent across most RDBMSs. Therefore, if you commit yourself to learning the underlying logic of SQL within the confines of any given RDBMS, then you will find your knowledge to be quite portable; that is, you will be able to quickly tailor your knowledge of SQL fundamentals to another RDBMS.

Queries, Statements, Clauses, and Keywords

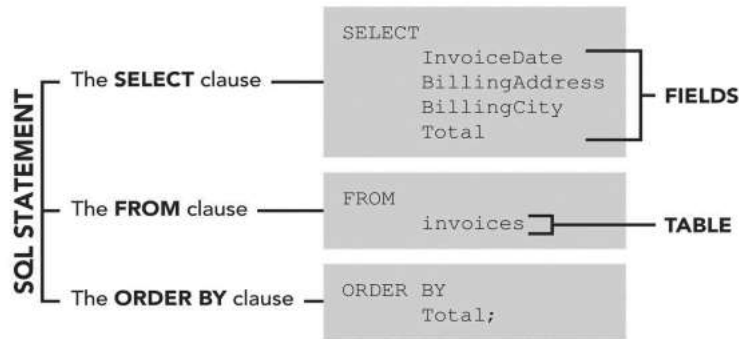
If you have had any previous experience with SQL, you may have heard these four words used interchangeably: query, statement, clause, and keyword. SELECT is a special keyword in SQL, but it is also referred to as a SELECT statement, the SELECT clause, or a SELECT query. So what is the difference? Let's start with the broadest term and finish with the most specific.

In its most basic form, a query is a request that returns information from the database in the form of records. A query can be composed of several SQL statements (which we will encounter in chapter 8 in the form of

subqueries). An SQL statement is any valid piece of code that is executed by the RDBMS. The code examples we just compared are both valid SQL

statements (since the RDBMS allows us to execute them) and queries (since they return a record set). A **clause** is a subsection of a query, containing at least one **keyword** and the relevant information to be used in conjunction with that keyword (in this case references to fields and tables).

↓ GRAPHIC fig. 21



Words in ALL CAPS are SQL keywords.

As you can see in Figure 21, an SQL statement can be comprised of many clauses, each containing at least one keyword, as well as references to fields and tables.

NOTE

As with the previous example, Figure 21 is both a complete SQL statement and a complete query. A query can contain multiple clauses, each starting with a keyword.

Introducing SQLite

Now that we have a basic grasp of database architecture and how we interact with it, let's switch gears a bit and move toward a more hands-on learning approach using real-world problem-solving scenarios. As we mentioned, there are several different RDBMSs to choose from. It would be ineffective in this context to try to go function by function, documenting the nuances of each individual RDBMS. Instead, we have selected SQLite as the official RDBMS of this book. SQLite is a very accessible and practical choice for new learners. It is an open-source software product and thus free to use for any purpose. About 99 percent of what you learn using

SQLite is applicable to most other RDBMSs. SQLite is also one of the most widely used RDBMS systems in the world—used in computers, mobile devices, even automobiles.² More information and some additional documentation can be found at <https://www.sqlite.org/index.html>.

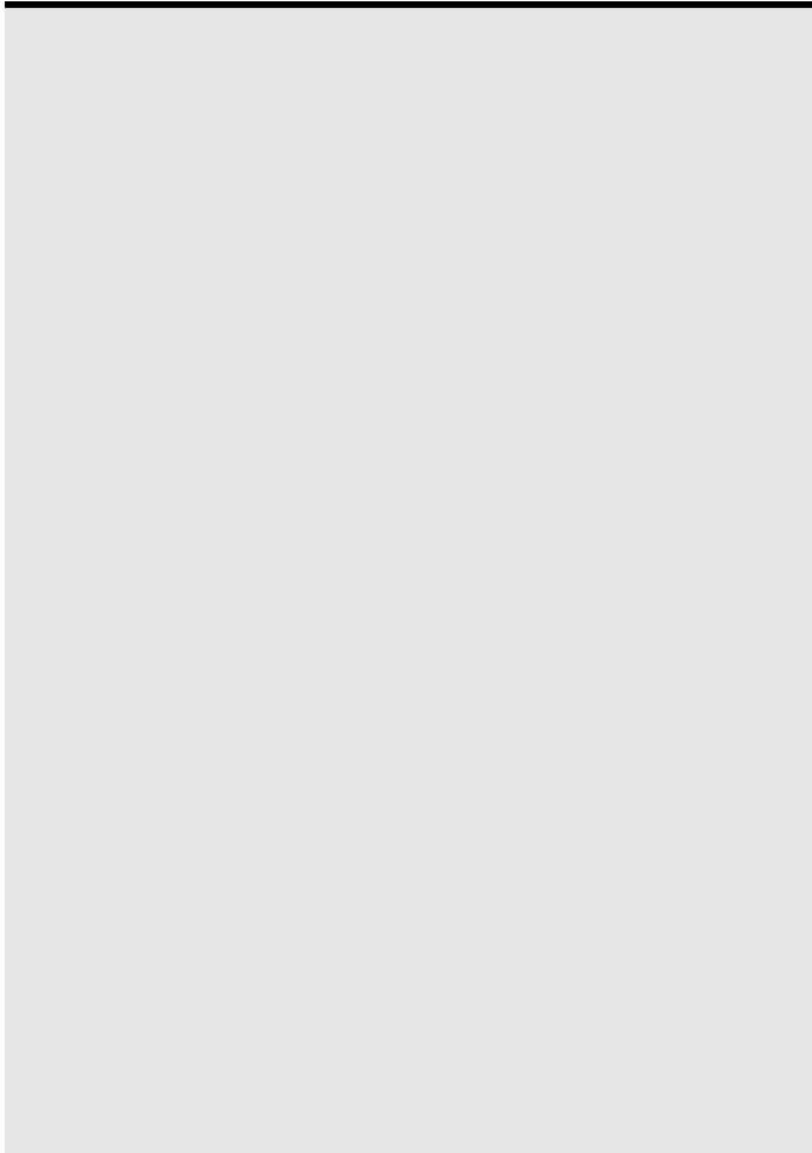
↓ **GRAPHIC** fig. 22



Some of the well-known companies that use SQLite.

 NOTE

The "Lite" in SQLite does not refer to the capabilities of this software, but rather to the fact that it is lightweight when it comes to setup complexity, administrative overhead, and resource usage.



Chapter Recap

»A table is a two-dimensional grid of rows and columns that contains data.

» Data can exist as a variety of different data types, such as strings of text, numbers, or special characters.

» Metadata describes the nature and format of the data, including any minimum/maximum character length or required numbers, letters, or special characters.

» Relational databases can contain many tables. Each table in a relational database should have a primary key that serves as a unique identifier for that table.

» A foreign key is any column in a table that exists as primary key in another table.

» The relationship between tables and their primary and foreign keys is called the database schema and can be shown visually by an entity relationship diagram (ERD), which functions as a blueprint for the database.

» There are a variety of different relational database management systems (RDBMSs) such as Oracle Database, Microsoft SQL Server, MySQL, IBM Db2, and SQLite. Though they differ in many ways, all share the structured query language as a backbone.

» The **SELECT** keyword is the most common SQL command used in SQL queries.

» SQL statements can contain multiple clauses using different SQL keywords.

» This text will use SQLite, but the skills you learn here can easily translate to other RDBMS platforms.

